Solution

General remarks:

- The number of ★'s gives a rough indicator of the task's difficulty. You should aim to solve at least all tasks with at most three ★'s. Tasks with more stars can be hard or require mathematical background that you should have from previous courses.
- You can always ask us for feedback or help during the exercise class.
- We recommend that you first read all tasks during class. Make sure that you have a rough approach for every task in mind before you start working on the details. Ask for help if a task is unclear such that you do not get stuck when we are not there to help.

1 Programming with Program Graphs (★)

Specify a program graph **PG** modelling a function that computes the greatest common divisor of two non-negative integers (given by variables n and m) and stores the result in a variable called `out`.

You can only use assignments with simple arithmetic operations (sum, multiplication, substraction), for example `x := x + y`, and simple Boolean tests, for example `x > 0`, as actions. In particular, computing the modulo or the factorial with an action is not allowed.

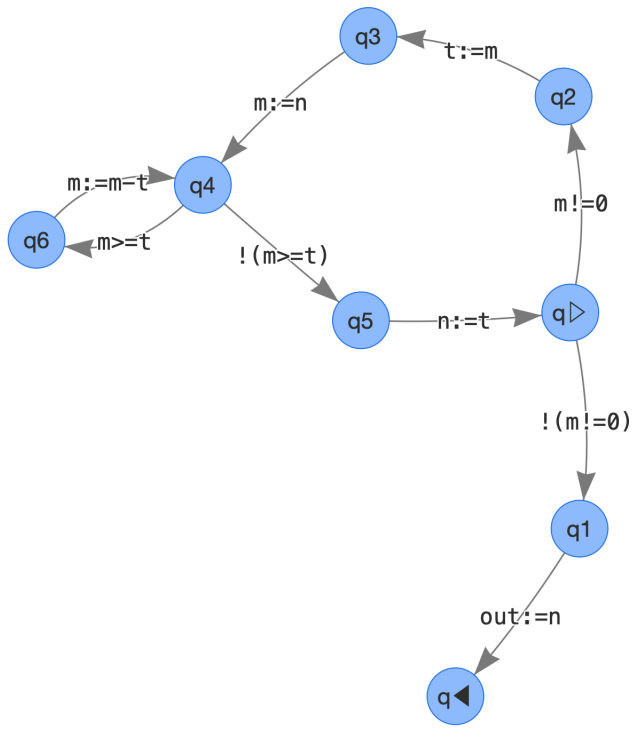
Hint: You may use the following pseudo-code for computing the greatest common divisor as inspiration. However, notice that you are not allowed to use the remainder ($a \mathtt{mod} b$) in an action; you may want to write a program graph for the remainder first.

```
function gcd(a, b)
    while b != 0
        t := b
        b := a mod b
        a := t
    return a
```

Hint 2: Try to design your program with a PG first. You can then try fm4fun, by writing the program in GCL and inspecting the PG that that the tool provides. You can then use fm4fun to run your PG and test it.

Solution

A possible program graph is depicted below. Note that we implement the modulo operation in nodes q_4 and q_6 using a loop.



A possible GCL program that generates the same PG is as follows:

```
do m != 0 ->
       t := m ;
       m := n ;
       do
       m >= t -> m:=m-t
       od ;
       n := t
od;
out := n
```

2 Semantic Functions (★, ★)

Define the semantic function $\mathcal{S}$ for all actions that you used in your program graph in task 1. For example, if your program includes as actions $\mathtt{m} \neq \mathtt{n}$ and $\mathtt{m} := \mathtt{n}$ you would need to define their semantic function as follows:

$$\begin{aligned}
\mathcal{S}[\![\mathtt{m} \neq \mathtt{n}]\!](\sigma) &= \begin{cases} \sigma, & \text{if } \sigma(m) \neq \sigma(n) \\ \text{undefined}, & \text{otherwise} \end{cases} \\
\mathcal{S}[\![\mathtt{m} := \mathtt{n}]\!](\sigma) &= \sigma[m \mapsto \sigma(n)]
\end{aligned}$$

Solution

The following solution is for the PG provided in the solution of the previous section. Your own solution may be different since you may be using different actions.

$$\begin{aligned}
\mathcal{S}[\![\mathtt{m} \neq \mathtt{0}]\!](\sigma) &= \begin{cases} \sigma, & \text{if } \sigma(m) \neq 0 \\ \text{undefined}, & \text{if } \sigma(m) = 0 \end{cases} \\
\mathcal{S}[\![\neg(\mathtt{m} \neq \mathtt{0})]\!](\sigma) &= \begin{cases} \sigma, & \text{if } \sigma(m) = 0 \\ \text{undefined}, & \text{if } \sigma(m) \neq 0 \end{cases} \\
\mathcal{S}[\![\mathtt{m} \geq \mathtt{t}]\!](\sigma) &= \begin{cases} \sigma, & \text{if } \sigma(m) \geq \sigma(t) \\ \text{undefined}, & \text{if } \sigma(m) < \sigma(t) \end{cases} \\
\mathcal{S}[\![\neg(\mathtt{m} \geq \mathtt{t})]\!](\sigma) &= \begin{cases} \sigma, & \text{if } \sigma(m) < \sigma(t) \\ \text{undefined}, & \text{if } \sigma(m) \geq \sigma(t) \end{cases} \\
\mathcal{S}[\![\mathtt{t} := \mathtt{m}]\!](\sigma) &= \sigma[t \mapsto \sigma(m)] \\
\mathcal{S}[\![\mathtt{m} := \mathtt{n}]\!](\sigma) &= \sigma[m \mapsto \sigma(n)] \\
\mathcal{S}[\![\mathtt{m} := \mathtt{m}\text{-}\mathtt{t}]\!](\sigma) &= \sigma[m \mapsto \sigma(m) - \sigma(t)] \\
\mathcal{S}[\![\mathtt{n} := \mathtt{t}]\!](\sigma) &= \sigma[n \mapsto \sigma(t)] \\
\mathcal{S}[\![\mathtt{out} := \mathtt{n}]\!](\sigma) &= \sigma[out \mapsto \sigma(n)]
\end{aligned}$$

3 Operational Semantics (★★)

Test whether your program graph from task 1 indeed computes the greatest common divisor by computing a complete execution sequence

$$\langle q_{\triangleright}; \sigma \rangle \quad \Longrightarrow^* \langle q_{\blacktriangleleft}; \sigma' \rangle$$

with $\sigma(n) = 24$ and $\sigma(m) = 8$. What is the sequence ω of executed actions? What is σ'?

Hint: Try to first write the sequence using pen and paper. Once you are confident that you know how to construct the sequence, use fm4fun to generate the sequence for the GCL program that corresponds to your PG. Compare the results to verify your answer.

Solution

A detailed execution sequence is given by the table below. The sequence of executed actions ω is shown in the leftmost column (read from top to bottom). The final memory is given in the last row: $\sigma(m) = 0$, $\sigma(n) = 8$, $\sigma(t) = 8$ and $\sigma(out) = 8$.

Action	Node	m	n	t	out
	q▷	8	24	0	0
`m!=0` ⟹	q2	8	24	0	0
`t:=m` ⟹	q3	8	24	**8**	0
`m:=n` ⟹	q4	**24**	24	8	0
`m>=t` ⟹	q6	24	24	8	0
`m:=m-t` ⟹	q4	**16**	24	8	0
`m>=t` ⟹	q6	16	24	8	0
`m:=m-t` ⟹	q4	**8**	24	8	0
`m>=t` ⟹	q6	8	24	8	0
`m:=m-t` ⟹	q4	**0**	24	8	0
`!(m>=t)` ⟹	q5	0	24	8	0
`n:=t` ⟹	q▷	0	**8**	8	0
`!(m!=0)` ⟹	q1	0	8	8	0
`out:=n` ⟹	q◀	0	8	8	**8**

4 Properties of Program Graphs (★★★)

1. Does your program graph from task 1 and its semantic function constitute a deterministic system? Justify your answer.
2. Does your program graph from task 1 and its semantic function constitute an evolving system? Justify your answer.

Solution

Yes, the program graph in the solution of 1.1 together with the semantic function in the solution of 1.2 constitutes both a deterministic and an evolving system. To check these properties, we use the sufficient criteria presented in the lecture (Propositions 1.22 and 1.25 in the FM book).

Deterministic system. By Proposition 1.22, it suffices to show that

$$\forall (q, \alpha_1, q_1), (q, \alpha_2, q_2) \in \mathbf{E}: \quad (\alpha_1, q_1) \neq (\alpha_2, q_2) \text{ implies } \mathrm{dom}(\mathcal{S}[\![\alpha_1]\!]) \cap \mathrm{dom}(\mathcal{S}[\![\alpha_2]\!]) = \emptyset.$$

Since the above criterion immediately holds for nodes with at most one outgoing edge, it suffices to consider the edges of two nodes: $q_\triangleright$ and q_4.

- There are two edges with source node $q_\triangleright$: $(q_\triangleright, \mathtt{m} \neq \mathtt{0}, q_2)$ and $(q_\triangleright, \neg(\mathtt{m} \neq \mathtt{0}), q_1)$. Computing the domains of the semantic functions for the actions of these actions yields:
  - $\mathrm{dom}(\mathcal{S}[\![\mathtt{m} \neq \mathtt{0}]\!]) = \{\sigma \mid \sigma(m) \neq 0\}$ and
  - $\mathrm{dom}(\mathcal{S}[\![\neg(\mathtt{m} \neq \mathtt{0})]\!]) = \{\sigma \mid \sigma(m) = 0\}$

  Clearly, $\mathrm{dom}(\mathcal{S}[\![\mathtt{m} \neq \mathtt{0}]\!]) \cap \mathrm{dom}(\mathcal{S}[\![\neg(\mathtt{m} \neq \mathtt{0})]\!]) = \emptyset$.
- There are two edges with source node q_4: $(q_4, \mathtt{m} \geq \mathtt{t}, q_6)$ and $(q_4, \neg(\mathtt{m} \geq \mathtt{t}), q_5)$. Computing the domains of the semantic functions for the actions of these actions yields:
  - $\mathrm{dom}(\mathcal{S}[\![\mathtt{m} \geq \mathtt{t}]\!]) = \{\sigma \mid \sigma(m) \neq \sigma(t)\}$ and
  - $\mathrm{dom}(\mathcal{S}[\![\neg(\mathtt{m} \geq \mathtt{t})]\!]) = \{\sigma \mid \sigma(m) = \sigma(t)\}$

  Clearly, $\mathrm{dom}(\mathcal{S}[\![\mathtt{m} \geq \mathtt{t}]\!]) \cap \mathrm{dom}(\mathcal{S}[\![\neg(\mathtt{m} \geq \mathtt{t})]\!]) = \emptyset$.

Hence, the system is deterministic.

Evolving system. By Proposition 1.25, it suffices to show that

$$\forall q \in \mathbf{Q} \setminus \{q_{\blacktriangleleft}\}\ \forall \sigma \in \mathbf{Mem}\ \exists (q, \alpha, q') \in \mathbf{E} \colon \sigma \in \mathsf{dom}(\mathcal{S}[\![\alpha]\!]).$$

We check the above criterion by computing the union of the domains of the actions of all outgoing edges for every node except the final one and checking whether this union equals the set of all memories. The following table summarizes these computations.

$q \in \mathbf{Q} \setminus \{q_{\blacktriangleleft}\}$	$\bigcup_{(q,\alpha,q') \in \mathbf{E}} \mathsf{dom}(\mathcal{S}[\![\alpha]\!])$	$= \mathbf{Mem}$?
$q_{\triangleright}$	$\{\sigma \mid \sigma(m) \neq 0\} \cup \{\sigma \mid \sigma(m) = 0\}$	✓
q_1	**Mem**	✓
q_2	**Mem**	✓
q_3	**Mem**	✓
q_4	$\{\sigma \mid \sigma(m) \neq \sigma(t)\} \cup \{\sigma \mid \sigma(m) = \sigma(t)\}$	✓
q_5	**Mem**	✓
q_6	**Mem**	✓

Since we obtain the full set of memories for every node except the final one, we conclude that, by Proposition 1.25, the system is evolving.

5 More on Semantics and Memories (★★★)

Let A and B be arrays corresponding to two vectors of length n and m, respectively. Construct a program graph for calculating the result of :

$$\sum_{i=0}^{n-1} \sum_{j=0}^{m-1} \mathsf{A}[i] \cdot \mathsf{B}[j]; \text{ and}$$

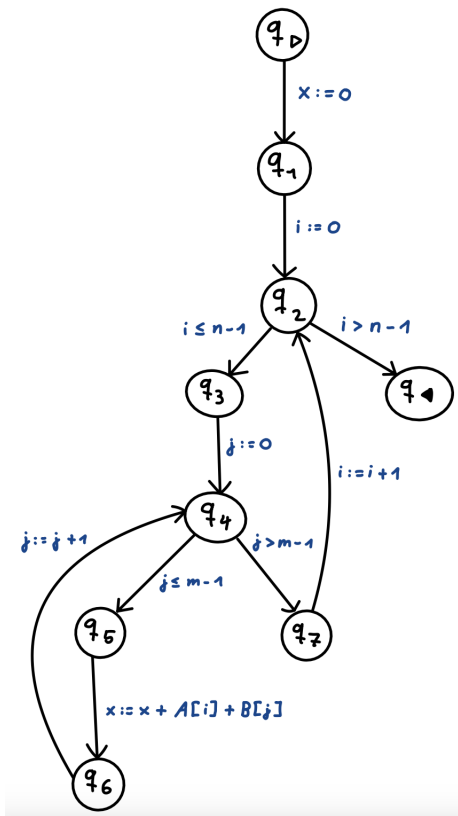
Moreover, define the semantic function for the involved actions in your PG.

Hint 1: As in previous exercises, you may want to use fm4fun to write the program in GCL and compare it with your PG.

Hint 2: Section 1.3 of the FM book explains how to define memories for arrays.

Solution 1

A possible PG could be



And a possible GCL code for it could be

```
x:=0;
i:=0;
do i<=(n-1) ->
  j:=0;
  do j<=(m-1) ->
    x:=x+A[i]+B[j];
    j:=j+1
  od;
  i := i+1
od
```

Act is the set of all actions (in blue) in the above program graph. The set of variables is **Var** $= \{\mathtt{x}, \mathtt{i}, \mathtt{j}, \mathtt{n}, \mathtt{m}\}$; the set of arrays is **Arr** $= \{A, B\}$. The set of memories is given by

$$\textbf{Mem} = (\textbf{Var} \cup \{Z[k] \mid Z \in \textbf{Arr}, 0 \leq k < \mathtt{length}(Z)\}) \ \rightarrow \ \textbf{Int}.$$

The semantics function

$$\mathcal{S}[\![.]\!] \colon \textbf{Act} \to (\textbf{Mem} \rightharpoonup \textbf{Mem})$$

is defined by the following table:

$\alpha \in$ **Act**	$\mathcal{S}[\![\alpha]\!](\sigma)$
`x := 0`	$\sigma[\mathtt{x} \mapsto 0]$
`i := 0`	$\sigma[\mathtt{i} \mapsto 0]$
`j := 0`	$\sigma[\mathtt{j} \mapsto 0]$
`i := i+1`	$\sigma[\mathtt{i} \mapsto \sigma(\mathtt{i}) + 1]$
`j := j+1`	$\sigma[\mathtt{j} \mapsto \sigma(\mathtt{j}) + 1]$
$\mathtt{i} \leq \mathtt{n}\text{-}1$	$\begin{cases} \sigma, & \text{if } \sigma(\mathtt{i}) \leq \sigma(\mathtt{n}) - 1 \\ \text{undefined}, & \text{otherwise} \end{cases}$
$\mathtt{i} > \mathtt{n}\text{-}1$	$\begin{cases} \sigma, & \text{if } \sigma(\mathtt{i}) > \sigma(\mathtt{n}) - 1 \\ \text{undefined}, & \text{otherwise} \end{cases}$
$\mathtt{j} > \mathtt{m}\text{-}1$	$\begin{cases} \sigma, & \text{if } \sigma(\mathtt{j}) > \sigma(\mathtt{m}) - 1 \\ \text{undefined}, & \text{otherwise} \end{cases}$
$\mathtt{j} \leq \mathtt{m}\text{-}1$	$\begin{cases} \sigma, & \text{if } \sigma(\mathtt{j}) \leq \sigma(\mathtt{m}) - 1 \\ \text{undefined}, & \text{otherwise} \end{cases}$
`x := x+A[i]+B[j]`	$\begin{cases} \sigma[\mathtt{x} \mapsto w], & \text{if } u = \sigma(\mathtt{i}), v = \sigma(\mathtt{j}), \\ & \quad \{A[u], B[v]\} \subseteq \mathsf{dom}(\sigma), \\ & \quad w = \sigma(\mathtt{x}) + \sigma(A[u]) + \sigma(B[v]) \\ \text{undefined}, & \text{otherwise} \end{cases}$